Tema 2
Estructuras Lineales
Listas, Pilas y Colas

---

1. Repaso de Apuntadores

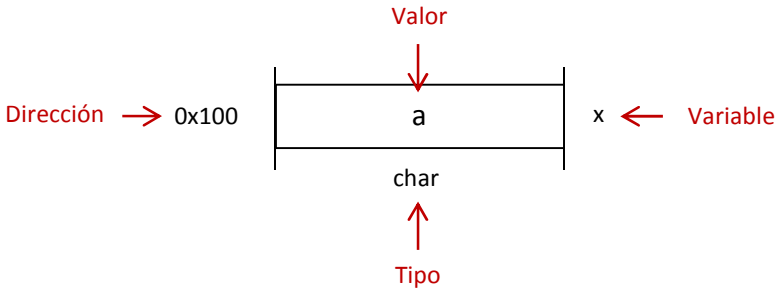
1.1. Direcciones de Memoria

La memoria principal de un computador consiste de un gran número de posiciones secuenciales de almacenamiento denominadas celdas de memoria, cada una de 8 bits (1 byte). Cada byte de memoria tiene un número único asociado, denominado dirección de memoria.

1.2. Las Variables y la Memoria

Cuando se declara una variable se le asocian tres atributos fundamentales a la misma:

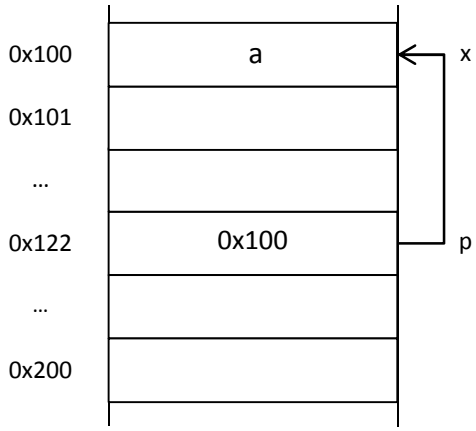
a) Nombre
b) Tipo
c) Dirección de memoria



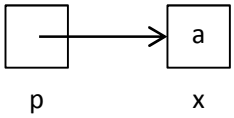
Al declarar una variable en un lenguaje de programación, implícitamente se está dando la orden de que se reserve un espacio contiguo de memoria lo suficientemente grande para guardar un valor del tipo especificado en la declaración, es decir, una cantidad fija de bytes.

1.3. Definición de Apuntador

Un apuntador es una variable que contiene una dirección de memoria. Normalmente, esa dirección es la posición en memoria de otra variable. Si una variable contiene la dirección de otra variable, entonces se dice que la primera variable apunta a la segunda. También se dice que un apuntador hace referencia a otra variable.



Gráficamente:



Declaración

Sintaxis
Apuntador a Nombre_de_Clase: p

Inicialización

a) Creación de un apuntador nulo:



Sintaxis
ptr ← **nulo** o ptr.construir()

b) Creación de un objeto de la clase a la cual hace referencia el apuntador:

Sintaxis
ptr.crear(Nombre_de_Clase)

c) Asignación del valor de otro apuntador.

Sintaxis
Apuntador a Nombre_de_Clase: p, q
p.crear(Nombre_de_Clase)
q ← p

d) Asignación de la dirección de memoria de una variable existente de la clase a la cual hace referencia el apuntador.

Sintaxis
Nombre_de_Clase: x
Apuntador a Nombre_de_Clase: p
P ← βx

1.4. Operaciones

Referenciación (β)

La operación de referenciación consiste en obtener la dirección de memoria correspondiente a una variable. Esta operación se representa con el operador β (beta).

Desreferenciación (↑)

La operación de desreferenciación o indirección consiste en obtener el valor de la variable a la cual apunta un apuntador. Esta operación se representa con el operador ↑.

Sintaxis
Nombre_de_Clase: x
Apuntador a Nombre_de_Clase: p
p.crear(Nombre_de_Clase)
P↑ ← x

1.5. Creación y Eliminación Dinámica de Objetos

Reservar Memoria: p.crear(Nombre_de_Clase)
Liberar Memoria: p.destruir()

2. Estructuras Lineales

Las estructuras lineales son aquellas en las cuales la relación existente entre los elementos de la estructura es de uno a uno, es decir un elemento solamente puede estar relacionado únicamente con otro elemento.

3. Lista (List)

Una lista es una secuencia de cero o más elementos de un tipo determinado (Aho, 1983). Una posible notación para representar una lista es la siguiente:

$$L = < e_1, e_2, e_3, \ldots, e_n >$$

Donde $n \geq 0$ es la cantidad de elementos de la lista y e_i ($0 \leq i \leq n$) es un elemento de la lista.

A la cantidad n de elementos de la lista se le denomina longitud. Si $n = 0$ se dice que la lista es vacía y se denota $L = <>$. Si $n > 0$, entonces se dice que e_1 es el primer elemento y e_n es el último elemento. La posición de un elemento e_i dentro de la lista es el lugar ocupado por dicho elemento dentro de la secuencia, es decir, i.
Una propiedad importante de una lista es que sus elementos pueden ser ordenados linealmente de acuerdo a la posición que ocupen en la lista; se puede decir que e_i es el predecesor de e_{i+1} para todo $i = 1, 2, 3, \ldots, n - 1$, y e_i es el sucesor de e_{i-1} para todo $i = 2, 3, 4, \ldots, n$.

3.1. Modelado de la Clase Lista

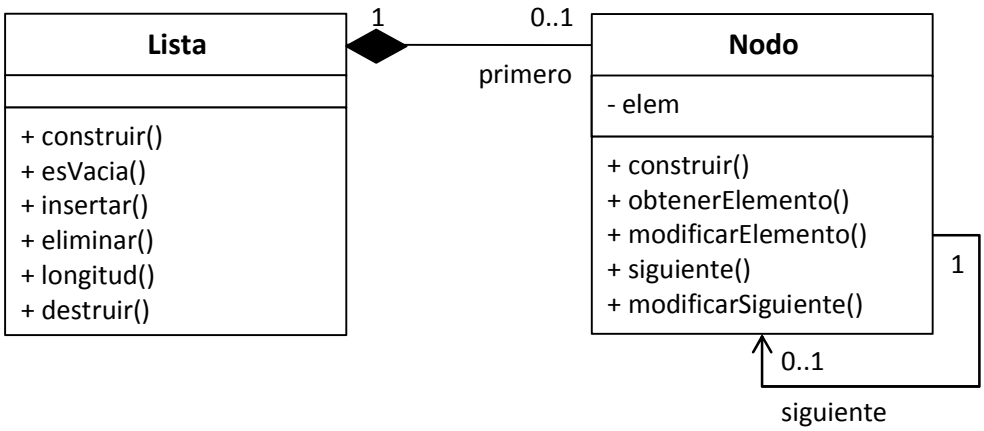


Figura 1. Diagrama de Clases de una lista enlazada.

3.2. Implementación de la Clase Lista

La representación dinámica de lista se denomina lista enlazada (*Linked List*), la cual es una colección o secuencia de elementos dispuestos uno detrás de otro, en la que cada elemento se conecta al siguiente por medio de un enlace o apuntador. Estos elementos son llamados nodos y se componen de dos campos:

a) Un campo para la Información, el cual es de tipo genérico.
b) Un apuntador al siguiente nodo en la lista.

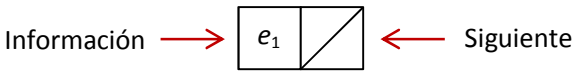


Figura 2. Nodo.

```
clase Nodo
      atributos:
            privado:
                  Elemento: elem
                  Apuntador a Nodo: sig
      métodos:
            público:
```

```
                proc construir()
                func obtenerElemento(): Elemento
                proc modificarElemento(Elemento: e)
                func siguiente(): Apuntador a Nodo
                proc modificarSiguiente(Apuntador a Nodo: sig)

fclase

proc Nodo::construir()
inicio
      instancia.elem.construir()
      instancia.sig ← nulo
fproc

func Nodo::obtenerElemento(): Elemento
inicio
      retornar(instancia.elem)
ffunc

proc Nodo::modificarElemento(Elemento: e)
inicio
      instancia.elem ← e
fproc

func Nodo::siguiente(): Apuntador a Nodo
inicio
      retornar(instancia.sig)
ffunc
proc Nodo::modificarSiguiente(Apuntador a Nodo: sig)
inicio
      instancia.sig ← sig
fproc
```

Al enlazar varios nodos, cada uno con el siguiente, se obtiene la lista. Gráficamente los enlaces se representan mediante flechas, pero hay que tomar en cuenta que son apuntadores y cada uno tiene la dirección de memoria de donde se encuentra el siguiente nodo en la lista.

Existen diferentes formas de representar una lista, siendo la más general la siguiente:

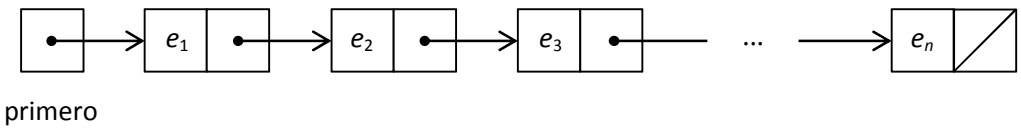


Figura 3. Representación general de lista simplemente enlazada.

En este caso el objeto *primero* es un **Apuntador a** Nodo el cual en un principio, cuando la lista está vacía, debe ser **nulo**. Al ir agregando elementos, este apuntador hará referencia al primer nodo de la lista enlazada. La implementación en pseudoformal de la lista enlazada queda como sigue:

```
// Clase Lista
clase Lista
      atributos:
            privado:
                  Apuntador a Nodo: primero
      métodos:
            público:
                  proc construir() // Crea una lista vacía
                  func esVacia(): Lógico
                  func longitud(): Entero
                  proc insertar(Elemento: e; Entero: posicion)
                  func consultar(Entero: posicion): Elemento
                  proc eliminar(Entero: posicion)
                  proc destruir()
fclase

// Constructor de la Lista
proc Lista::construir()
inicio
      instancia.primero ← nulo
fproc

func Lista::esVacia(): Lógico
inicio
      retornar(instancia.primero = nulo)
ffunc
func Lista::longitud(): Entero
var
      Apuntador a Nodo: sig
      Entero: i
inicio
      si(instancia.primero = nulo) entonces
            retornar(0)
      sino
            i ← 1
            sig ← instancia.primero↑.siguiente()
            mientras(sig ≠ nulo) hacer
                  i ← i + 1
                  sig↑.siguiente()
            fmientras
            retornar(i)
      fsi
ffunc
```

```
proc Lista::insertar(Elemento: e; Entero: posicion)
var
      Apuntador a Nodo: ant, sig, nuevo
      Entero: i
inicio
      nuevo.crear(Nodo)
      nuevo↑.construir()
      nuevo↑.modificarElemento(e)
      si(pos = 1) entonces    // Insertar al principio
            nuevo↑.modificarSiguiente(instancia.primero)
            instancia.primero ← nuevo
      sino                    // Insertar en el centro o final
            ant ← instancia.primero
            sig ← ant↑.siguiente()
            i ← 2
            mientras(i < posicion ∧ sig ≠ nulo) hacer
                  ant ← sig
                  sig ← sig↑.siguiente()
                  i ← i + 1
            fmientras
            ant↑.modificarSiguiente(nuevo)
            nuevo↑.modificarSiguiente(sig)
      fsi
fproc

func Lista::consultar(Entero: posicion): Elemento
var
      Apuntador a Nodo: ant, act
      Entero: i
inicio
      si(posicion = 1) entonces
            retornar(instancia.primero↑.obtenerElemento())
      else
            ant ← instancia.primero
            act ← ant↑.siguiente()
            i ← 2
            mientras(i < posicion ∧ act ≠ nulo) hacer
                  ant ← ant↑.siguiente()
                  act ← ant↑.siguiente()
                  i ← i + 1
            fmientras
            si(i = posicion ∧ act ≠ nulo) entonces
                  retornar(act↑.obtenerElemento())
            fsi
      fsi
ffunc
```

```
proc Lista::eliminar(Entero: posicion)
var
      Apuntador a Nodo: ant, act, sig
      Entero: i
inicio
      si(posicion = 1) entonces
            act ← instancia.primero
            instancia.primero ← act↑.siguiente()
            act.destruir()
      sino
            ant ← instancia.primero
            act ← ant↑.siguiente()
            sig ← act↑.siguiente()
            i ← 2
            mientras(i < posicion ∧ sig ≠ nulo) hacer
                  ant ← act
                  act ← sig
                  sig ← sig↑.siguiente()
                  i ← i + 1
            fmientras
            si(i = posicion) entonces
                  ant↑.modificarSiguiente(sig)
                  act.destruir()
            fsi
      fsi
fproc

// Destructor de la Lista
proc Lista::destruir()
var
      Apuntador a Nodo: act
inicio
      mientras(instancia.primero ≠ nulo) hacer
            act ← instancia.primero
            instancia.primero ← instancia.primero↑.siguiente()
            act.destruir()
      fmientras
fproc
```

Otra representación un poco más completa de lista enlazada agrega un apuntador al último elemento de la lista para facilitar inserciones al final y además incluye un valor entero con la cantidad de elementos en la lista, lo cual logra que el cálculo de la longitud baje de una complejidad O(n) a O(1).

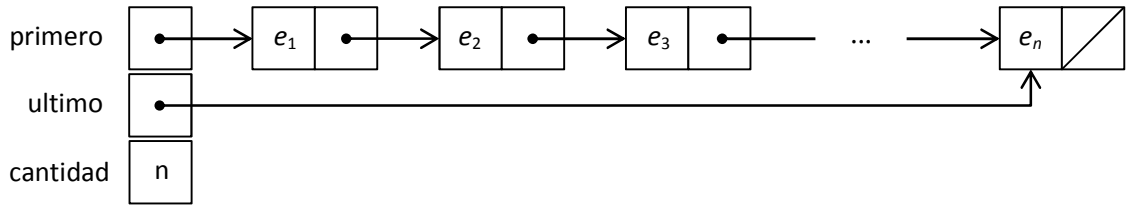


Figura 4. Otra representación de lista simplemente enlazada.

```
clase Lista
      atributos:
            privado:
                  Entero: cantidad
                  Apuntador a Nodo: primero, ultimo
      métodos:
            público:
                  proc construir()
                  func esVacia(): Logico
                  func longitud(): Entero
                  proc insertar(Elemento: e; Entero: posicion)
                  func consultar(Entero: posicion): Elemento
                  proc eliminar(Entero: posicion)
                  proc destruir()
fclase

// Constructor de la Lista
proc Lista::construir()
inicio
      instancia.primero ← nulo
      instancia.ultimo ← nulo
      instancia.cantidad ← 0
fproc

func Lista::esVacia(): Lógico
inicio
      retornar(instancia.cantidad = 0)
ffunc
func Lista::longitud(): Entero
inicio
      retornar(instancia.cantidad)
ffunc

proc Lista::insertar(Elemento: e; Entero: posicion)
var
      Apuntador a Nodo: ant, sig, nuevo
inicio
      nuevo.crear(Nodo)
      nuevo↑.construir()
      nuevo↑.modificarElemento(e)
```

```
        si(instancia.cantidad = 0) entonces
                instancia.primero ← nuevo
                instancia.ultimo ← nuevo
        sino
                si(posicion = 1) entonces     // Insertar al principio
                        nuevo↑.modificarSiguiente(instancia.primero)
                        instancia.primero ← nuevo
                sino                          // Insertar al final
                        si(posicion = instancia.cantidad + 1) entonces
                                instancia.ultimo.modificarSiguiente(nuevo)
                                instancia.ultimo ← nuevo
                        sino                  // Insertar en el centro
                                ant ← instancia.primero
                                sig ← ant↑.siguiente()
                                para i ← 2 hasta pos – 1 hacer
                                        ant ← sig
                                        sig ← sig↑.siguiente()
                                fpara
                                ant↑.modificarSiguiente(nuevo)
                                nuevo↑.modificarSiguiente(sig)
                        fsi
                fsi
        fsi
        instancia.cantidad ← instancia.cantidad + 1
fproc
```

Ejercicios:

1. Realice los algoritmos correspondientes a los métodos consultar, eliminar y destruir de la segunda representación de lista enlazada.
2. Extienda la clase Lista con los siguientes métodos:
   a. **proc** modificar(Elemento: e; **entero**: posicion). O(n)
   b. **proc** insertarAlPrincipio(Elemento: e), inserta un elemento al principio de la lista. O(1).
   c. **proc** insertarAlFinal(Elemento: e), inserta un elemento al final de la lista. O(1).
   d. **proc** invertir(), invierte la lista. Debe tener complejidad máxima de O(n).

3.3. Tipos de Listas Enlazadas

Las listas enlazadas tienen diferentes representaciones dependiendo de la manera en la que sus nodos sean enlazados, por lo tanto pueden dividirse en las siguientes cuatro categorías:

Listas Simplemente Enlazadas

Las listas simplemente enlazadas tienen en cada nodo un único enlace al nodo siguiente. Estas se implementaron en la sección anterior y gráficamente se ven como sigue:

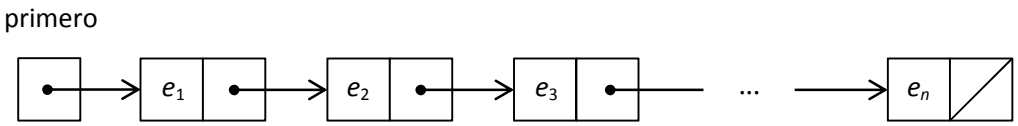


Figura 5. Representación de lista simplemente enlazada.

Listas Doblemente Enlazadas

Las listas doblemente enlazadas tienen en cada nodo dos enlaces: Un enlace al siguiente nodo y un enlace al anterior. Gráficamente se ven como sigue:

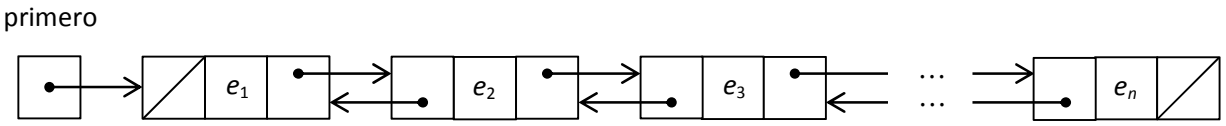


Figura 6. Representación de lista doblemente enlazada.

Listas Circulares Simplemente Enlazadas

Las listas circulares simplemente enlazadas tienen en cada nodo un único enlace al nodo siguiente al igual que las listas simplemente enlazadas, pero además su último nodo apunta al primero:

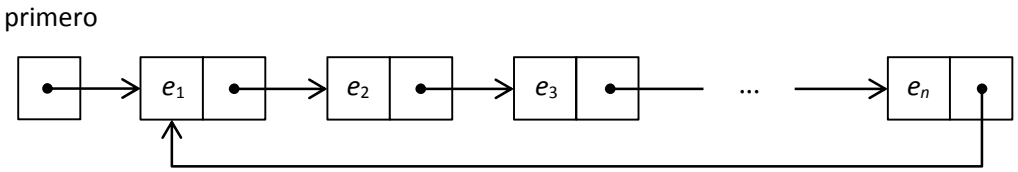


Figura 7. Representación de lista circular simplemente enlazada.

Listas Doblemente Enlazadas

Las listas circulares doblemente enlazadas tienen en cada nodo dos enlaces, un enlace al siguiente nodo y un enlace al anterior, pero además el último nodo tiene como siguiente al primero de la lista y el primer nodo de la lista tiene como anterior al último. Gráficamente se ven como sigue:

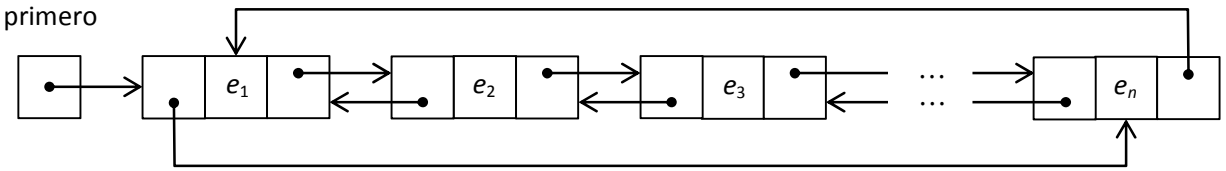


Figura 8. Representación de lista circular doblemente enlazada.

Ejercicios:

3. Implemente la clase NodoDoble (no olvide los métodos anterior() y modificarAnterior()).
4. Implemente las clases ListaDE, ListaCircular y ListaCircularDE.

4. Pila (Stack)

Es un tipo especial de lista en la cual todas las inserciones y eliminaciones se hacen en uno de los finales de la lista, llamado tope (Aho, 1983). En esta estructura el último elemento que llega es el primer elemento que sale, por lo cual también se llama estructura LIFO (Last In, First Out).

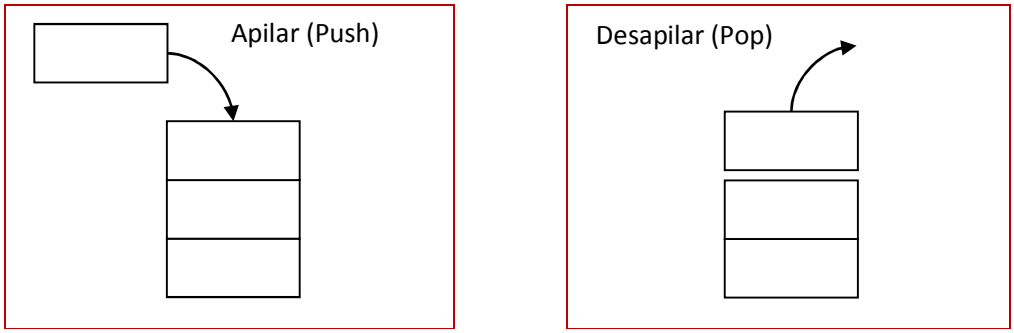


Figura 9. Movimientos válidos de una Pila

En todo momento el único elemento visible es el último elemento que se colocó, y el punto en el cual se encuentra se denomina tope. El lugar en el cual se encuentra el primer elemento que se insertó a la pila se denomina fondo. Si una pila no tiene elementos se dice que se encuentra vacía y no tiene sentido referirse ni a su tope ni a su fondo.

4.1. Implementación del TDA Pila

Para implementar la clase Pila se usará la clase Nodo que fue implementada previamente.

```
clase Pila
      atributos:
            privado:
                  Apuntador a Nodo: tope  // Clase Nodo implementada previamente
      métodos:
            público:
                  proc construir()        // Crea una pila vacía
                  func esVacia(): Lógico
                  proc apilar(Elemento: e)
                  proc desapilar()
                  func obtenerTope(): Elemento
                  proc destruir()
fclase
```

```
proc Pila::construir()
inicio
      instancia.tope ← nulo
fproc

func Pila::esVacia(): Lógico
inicio
      retornar(instancia.tope = nulo)
ffunc

proc Pila::apilar(Elemento: e)
var
      Apuntador a Nodo: nuevo
inicio
      nuevo.crear(Nodo)
      nuevo↑.modificarElemento(e)
      nuevo↑.modificarSiguiente(instancia.tope)
      instancia.tope ← nuevo
fproc

proc Pila::desapilar()
var
      Apuntador a Nodo: aux
inicio
      aux ← instancia.tope
      instancia.tope ← instancia.tope↑.siguiente()
      aux.destruir()
fproc

func Pila::obtenerTope(): Elemento
inicio
      retornar(instancia.tope↑.obtenerElemento())
ffunc
```

5. Cola (Queue)

Es un tipo especial de lista en la cual los elementos se insertan por un extremo y se eliminan por el otro (Aho, 1983). Debido a esta propiedad las colas también son llamadas estructuras FIFO (First In, First Out).
En todo momento, el único elemento visible de la estructura es el primero que se colocó, denominado frente o cabeza, y mientras éste no haya salido no es posible tener acceso al siguiente elemento. Si una cola no contiene elementos, se dice que se encuentra vacía, y no tiene sentido referirse a su frente.

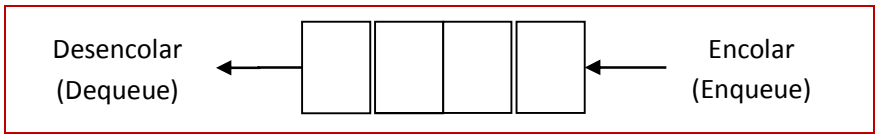


Figura 10. Movimientos válidos de una cola.

5.1. Implementación del TDA Cola

Para implementar la clase Cola se usará la clase Nodo que fue implementada previamente.

```
clase Cola
      atributos:
            privado:
                  Apuntador a Nodo: frente, final
      métodos:
            público:
                  proc construir()
                  func esVacia(): Lógico
                  proc encolar()
                  proc desencolar()
                  func obtenerFrente(): Elemento
                  proc destruir()
fclase

proc Cola::construir()
inicio
      instancia.frente ← nulo
      instancia.final ← nulo
fproc

func Cola::esVacia(): Lógico
inicio
      retornar(instancia.frente = nulo)
ffunc

proc Cola::encolar(Elemento: e)
var
      Apuntador a Nodo: nuevo
inicio
      nuevo.crear(Nodo)
      nuevo↑.construir()
      nuevo↑.modificarElemento(e)
      si(instancia.frente = nulo) entonces
            instancia.frente ← nuevo
      sino
            instancia.final↑.modificarSiguiente(nuevo)
      fsi
      instancia.final ← nuevo
fproc

proc Cola::desencolar()
var
      Apuntador a Nodo: aux
inicio
```

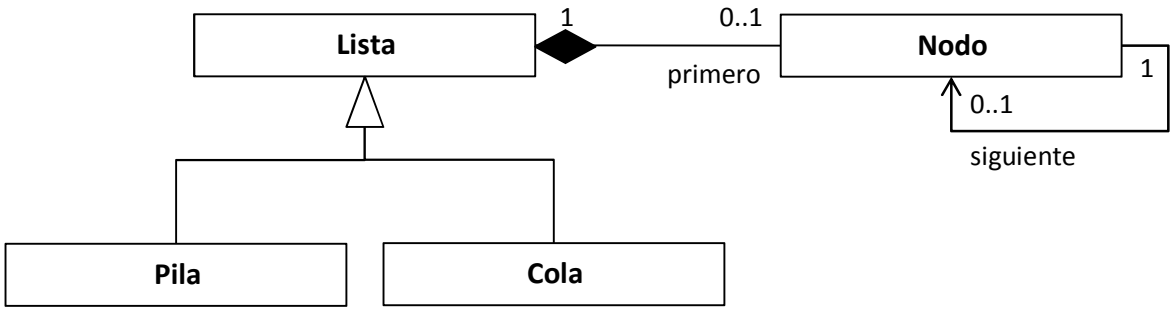
```
      aux ← instancia.frente
      instancia.frente ← instancia.frente↑.siguiente()
      aux.destruir()
fproc

func Cola::obtenerFrente(): Elemento
inicio
      retornar(instancia.frente↑.obtenerElemento())
ffunc
```

Ejercicios:

5. Implemente el método destruir de las clases Pila y Cola.
6. Implemente:
   a. Una operación que invierta una Pila usando una Cola.
   b. Una operación que invierta una Cola usando una Pila.

6. Pila y Cola como Especializaciones de Lista



Como las pilas y las colas son listas con un comportamiento más restringido sobre los elementos que las conforman se pueden modelar como especializaciones de la clase Lista, y heredar sus atributos y métodos.

6.1. Implementación de la Clase Pila como Especialización de la Clase Lista

```
// Clase Lista
clase Lista
      atributos:
            privado:
                  Entero: cantidad
                  Apuntador a Nodo: primero, ultimo
      métodos:
            público:
```

```
                  proc construir()
                  func esVacia(): Logico
                  func longitud(): Entero
                  virtual proc insertar(Elemento: e; Entero: posicion)
                  virtual func consultar(Entero: posicion): Elemento
                  virtual proc eliminar(Entero: posicion)
                  proc destruir()
fclase

// Implementación de los métodos de la clase Lista (...)

// Clase Pila
clase Pila
      hereda_de Lista excepto longitud
      sobrescribe insertar, consultar, eliminar
      métodos:
            privado:
                  // Cambio del modificador de control de acceso
                  proc insertar(Elemento: e; Entero: posicion)
                  func consultar(Entero: posicion): Elemento
                  proc eliminar(Entero: posicion)
            público:
                  // El método esVacia se hereda
                  proc construir()
                  proc apilar(Elemento: e)
                  proc desapilar()
                  func tope(): Elemento
                  proc destruir()
fclase

// Implementación de los métodos de la clase Pila
proc Pila::construir()
inicio
      super.construir()
fproc

proc Pila::apilar(Elemento: e)
inicio
      instancia.insertar(e, 1)
fproc
proc Pila::desapilar()
inicio
      instancia.eliminar(1)
fproc

func Pila::tope(): Elemento
inicio
      retornar(instancia.consultar(1))
ffunc
```

6.2. Implementación de la Clase Cola como Especialización de la Clase Lista

Ejercicio:

7. Implemente la clase Cola como especialización de la clase Lista.

7. Referencias

- Deitel, Harvey M. y Deitel, Paul J. (2004). Cómo programar en C/C++ y Java. Pearson Education.
- Joyanes A., L. y Zahonero M., I. (2001). Programación en C. Metodología, Algoritmos y Estructura de Datos. McGraw-Hill.
- Martínez, A. A. y Rosquete, D. H. (2009). NASPI: Una Notación Algorítmica Estándar para la Programación Imperativa. Télématique. 8(3). 55 – 74.
- Ottogalli, K., Martínez, A. y León L. (2011). NASPOO: Una Notación Algorítmica Estándar para la Programación Orientada a Objetos. Télématique. 10(1). 81 – 102.